

MOD-06: Advanced React — State at Scale & the Data Layer

Enterprise React Specialist Handbook · Principal Frontend Architect track, HCLSoftware

Sep 18, 2026 · @Someone

Chapter scope and standing conventions

MOD-05 gave a component memory, timing and a location. MOD-06 is where state **leaves the component** — and where an application stops being a collection of screens and becomes a system with a data layer, a failure model and a transport contract.

ID	Subtopic	Downstream module that breaks without it
MOD-06.1	Context API — provider model, blast radius, splitting	MOD-07.3 re-render profiling
MOD-06.2	Error boundaries and error pages	MOD-07.1 lazy loading — a chunk-load failure needs a boundary
MOD-06.3	Redux architecture — flow, reducers, middleware	MOD-06.4 (RTK is incoherent without it)
MOD-06.4	Redux Toolkit — slices, thunks, adapters, selectors	MOD-07.3 selector-driven render minimization
MOD-06.5	Axios — instances, interceptors, refresh queue	MOD-06.6 transport decision
MOD-06.6	Fetch in React — typed wrapper, abort, when to adopt a library	MOD-07.2 Suspense-based data
MOD-06.7	Generators and sagas	Long-running cancellable workflows

Standing conventions carried forward

1. **Strict TypeScript.** `strict: true`, zero `any`, no silencing casts. Errors are typed as discriminated unions, not `Error` or `unknown` passed around. Redux state, actions, thunks and selectors are fully typed; `RootState` and `AppDispatch` are inferred from the store, never hand-written.

2. **Jest + React Testing Library.** Slices and selectors tested as pure functions; interceptors tested against a mocked adapter with **concurrent** 401s; boundaries tested for capture, fallback and recovery; typed clients tested for abort.
3. **WCAG 2.1 AA.** Error boundary fallbacks carry `role="alert"`; background refreshes announce politely; every retry affordance is a real, keyboard-operable `<button>`.
4. **APP-01 — Enterprise Build Tooling and CI/CD Pipelines** remains appended after MOD-07.

One ordering decision worth stating

The syllabus lists "Redux Toolkit, Redux". This chapter deliberately inverts that: **MOD-06.3 teaches Redux principles first, MOD-06.4 teaches RTK second.** RTK's `createSlice` lets you write what looks like mutation (`state.value = x`) and produces an immutable update through Immer. That illusion is only safe once reducer purity and immutability are already internalized. A learner who meets RTK first concludes that Redux reducers mutate state, and will eventually mutate outside a slice — the non-deterministic DevTools defect in Part 2.

Source alignment: built against the learner's handwritten notes on unidirectional data flow (10/08/26), higher-order functions and the call/bind patterns (27/08/26), and the Spring MVC inversion-of-control and dependency-injection material (01/09/26), whose middleware and interception concepts map directly onto Redux middleware and Axios interceptors.

Prerequisite gate. MOD-01.8 (`reduce`), MOD-02.2 (higher-order functions), MOD-02.6 (`AbortController`) and MOD-05.4 (`useMemo` and reference identity) must answer cold.

Part 1 — High-level overview and architectural connections

Every subtopic in this module answers one question: **where does state live when it stops belonging to a component?** There are only four honest answers — the component itself, a Context, a store, or the server — and most enterprise React defects come from choosing the wrong one and then compensating with effects.

The seven subtopics

MOD-06.1 — Context API. A dependency-injection mechanism, not a state manager. It solves prop drilling by making a value available to a subtree. Its defining characteristic is the **blast radius**: when the provider's value changes identity, every consumer re-renders, regardless of which part of the value it actually reads. The architectural content is splitting stable identity from volatile values.

MOD-06.2 — Error boundaries and error pages. React's only mechanism for catching render-phase errors. Class components with `getDerivedStateFromError` and `componentDidCatch`, arranged in a tier hierarchy, with a triage model distinguishing a render crash from a 401, a 403, a 404 and a 500 — four different failures that need four different fallbacks.

MOD-06.3 — Redux architecture. A single store, plain-object actions, pure reducers, unidirectional flow, and a middleware pipeline. Taught as a set of constraints that make large state *inspectable*, which is the property the pattern actually buys.

MOD-06.4 — Redux Toolkit. `configureStore`, `createSlice`, `createAsyncThunk`, the Immer mutation illusion, `createEntityAdapter` for normalization, `createSelector` for memoized projections, and DevTools hygiene.

MOD-06.5 — Axios in the enterprise. Instance isolation per API surface, request and response interceptors, and the piece that separates a competent client from a naive one: a **concurrency-locked 401 refresh queue**.

MOD-06.6 — Fetch in React. The honest trade-off against Axios, a typed wrapper worth owning, aborting on unmount, and the decision point at which a team should stop hand-rolling server state and adopt RTK Query or TanStack Query.

MOD-06.7 — Generators and sagas. `function*`, `yield`, the iterator protocol, two-way communication, `yield*` delegation, and why a generator-driven flow is cancellable in a way `async / await` is not.

The runtime roots: three direct inheritances

Higher-order functions (MOD-02.2) drive both Redux middleware and Axios interceptors. Redux middleware has the signature `store => next => action => result` — a function returning a function returning a function, which is currying from MOD-02.2 applied to a dispatch pipeline. Each middleware wraps the next, forming a chain that an action passes through before reaching the reducer. Axios interceptors are the same idea with a flatter API: functions registered to wrap a request or response as it passes through. **Both are the decorator pattern built from closures**, and a learner who holds MOD-02.2 recognizes them immediately rather than memorizing two unrelated APIs. The learner's Spring notes on inversion of control describe the same architectural move in Java: the framework owns the pipeline, your code registers behaviour into it.

reduce (MOD-01.8) underpins entity adapters and normalized state. The `{ byId, allIds }` shape built by hand in MOD-01.8 is exactly what `createEntityAdapter` formalizes as `{ entities, ids }`. A reducer *is* a reduce function: `(state, action) => newState` is the same signature as `(accumulator, item) => newAccumulator`, and the

store's state is the accumulated result of every action ever dispatched. That is not an analogy — it is the literal implementation, and it is why `Array.prototype.reduce` was taught in the first module.

Error boundaries (MOD-06.2) must exist before code-split lazy loading (MOD-07.1). A lazily loaded chunk can fail to download: a deployment rotates the chunk hashes while a user has a stale `index.html`, a CDN edge misses, a corporate proxy blocks it, or the network drops mid-navigation. `React.lazy` surfaces that as a **thrown error during render**, which only an error boundary can catch. Ship code splitting without boundaries and the first deploy after a user leaves a tab open produces a white screen on navigation. This is why MOD-06.2 sits here rather than in MOD-07: the safety net must be installed before the trapeze.

Dependency topology

flowchart TD

```

A["MOD-06.1<br/>Context API"] --> B["MOD-06.3<br/>Redux architecture"]
B --> C["MOD-06.4<br/>Redux Toolkit"]
D["MOD-06.2<br/>Error boundaries"] --> E["MOD-06.5<br/>Axios + refresh queue"]
C --> E
E --> F["MOD-06.6<br/>Fetch and typed clients"]
F --> G["MOD-06.7<br/>Generators and sagas"]
C --> G
H["MOD-02.2<br/>Higher-order functions"] -.->|"middleware<br/>and interceptors"| B
H -.-> E
I["MOD-01.8<br/>reduce"] -.->|"reducers and<br/>entity adapters"| C
J["MOD-02.6<br/>AbortController"] -.-> F
K["MOD-05.4<br/>useMemo identity"] -.->|"provider value<br/>stability"| A
D ==>|"REQUIRED BEFORE"| L["MOD-07.1<br/>Lazy chunks"]
C --> M["MOD-07.3<br/>Selector-driven<br/>render minimization"]

```

The double arrow is the hard ordering constraint in this handbook: nothing in MOD-07.1 is safe to ship until MOD-06.2 is in place.

What this chapter defers

RTK Query is named in MOD-06.6 as the adoption target and its entity-adapter contract is built in MOD-06.4, but its full API is beyond syllabus scope. `useReducer` appears in MOD-06.3 as the local-scale illustration of the reducer pattern before the store arrives. Server Components and server actions are out of scope. Saga testing beyond the effect-primitive level is left to the team's own saga suite.

Part 2 — Production use cases and enterprise failure modes

MOD-06.1 — Context API

Use case. A tenant-and-theme provider wrapping the entire portal shell, consumed by a 2,000-cell job grid.

Failure mode: the blast radius. The provider builds its value inline — `value={{ tenant, theme, setTheme, searchQuery, setSearchQuery }}` — so the object has a new identity on every render. The search box lives in the same provider, so every keystroke changes provider state, produces a new value object, and re-renders **every consumer in the tree**: 2,000 cells, 40 widgets, the navigation, the breadcrumb. Typing in a filter box drops the page to single-digit frames per second. `React.memo` on the cells does not help, because context consumers re-render regardless of props. The fix is splitting volatile state out of the stable context and memoizing the value (MOD-05.4).

Failure mode: Context used as a state manager. A team puts all server data in Context because "Redux is boilerplate". They get no devtools, no time-travel, no middleware, no normalized cache and no selector-level subscription — every consumer re-renders on every change. The cost of Redux's boilerplate is paid once; the cost of Context-as-store is paid on every render, forever.

MOD-06.2 — Error boundaries

Use case. A third-party charting library rendering tenant telemetry inside a dashboard tile.

Failure mode: the white screen. The chart throws on a malformed data point — a `null` where a number was expected, in one tenant's data only. With no boundary, React unmounts the **entire application tree**, and the user sees a blank page with a console error they will never read. Support receives "the portal is broken" with no reproduction path, because it only happens for that tenant. A widget-tier boundary would have degraded one tile and kept the other 39 alive.

Failure mode: boundaries that catch nothing useful. Error boundaries do **not** catch errors in event handlers, in asynchronous callbacks, in `setTimeout`, in server-side rendering, or in the boundary itself. A team adds one boundary at the root, assumes it covers everything, and is surprised when an unhandled rejection in a click handler still reaches the window.

Failure mode: no triage. A single generic "Something went wrong" fallback for a 401, a 403, a 404 and a render crash. The 401 user needed re-authentication, the 403 user needed to know they lack permission, the 404 user needed navigation, and the crash needed a retry. All four get an apology and a dead end, and the support queue absorbs the difference.

Failure mode: silent telemetry. A boundary that renders a fallback but never reports means the error rate is invisible. The team believes the application is stable because nobody files tickets for a tile that quietly shows a placeholder.

MOD-06.3 — Redux architecture

Use case. A permissions matrix and a billing panel that must agree on the same user records.

Failure mode: the impure reducer. A reducer that calls `Date.now()`, `Math.random()` or `crypto.randomUUID()`, or dispatches from inside itself, breaks the guarantee that replaying the action log reproduces the state. DevTools time-travel produces a different result each time, so the tool the team bought Redux for stops working exactly when a hard bug needs it.

Failure mode: middleware that swallows. A logging middleware that forgets to `return next(action)` silently drops every action. The UI stops responding, nothing errors, and the cause is one missing `return` in a file nobody suspects.

Failure mode: everything in the store. Putting form input state, hover state and modal open/closed in Redux produces an action log so noisy that real state transitions are unreadable, and every keystroke round-trips through the middleware pipeline.

MOD-06.4 — Redux Toolkit

Use case. Normalizing tenants, users, roles and permissions across modules.

Failure mode: mutation outside Immer. `createSlice` reducers look mutable because Immer wraps them in a draft. A developer generalizes and mutates state in a **thunk**, a selector, or a plain non-RTK reducer — outside the draft, on the real state object. The store's reference does not change, subscribers do not fire, and the action log no longer describes the state. Time-travel becomes non-deterministic: stepping back and forward yields different results depending on which mutations happened to land. This is the most damaging Redux defect in existence because it destroys the debugging tool you would use to find it.

Failure mode: unnormalized nested state. The permissions module and the billing module each store their own copy of the user object. A name change updates one and not the other, so the same user appears under two names on two screens. Worse, memory and payload size grow with the duplication, and there is no single place to fix it.

Failure mode: selectors that allocate. `useSelector(state => state.tenants.ids.map(id => state.tenants.entities[id]))` returns a **new array every call**. `useSelector` compares with reference equality, so the component re-renders

on every store action, including ones from unrelated slices. The team concludes Redux is slow.

Failure mode: `createSelector` with unstable inputs. A memoized selector whose input selector returns a fresh object recomputes every time, so the memo is decorative. Cache size is 1 by default, so a selector called with two different arguments in the same render alternates and never hits.

MOD-06.5 — Axios

Use case. A dashboard issuing 12 parallel widget requests when a tenant's access token has just expired.

Failure mode: the refresh stampede. All 12 requests return 401. A naive interceptor calls the refresh endpoint on each one: 12 concurrent refresh calls. Most refresh-token implementations rotate the token and invalidate the previous one, so the first refresh succeeds and the other 11 present a now-invalid token, get 401 back, and trigger another refresh each. The tab issues hundreds of requests, the auth service rate-limits the tenant, and every user in that tenant is logged out.

Failure mode: the infinite 401 loop. The interceptor retries a failed request without marking it as retried. The retry gets 401, the interceptor retries again, forever. The browser tab freezes; the server sees a request flood from one client. A `_retry` flag on the config is the one line that prevents it.

Failure mode: the shared global instance. Using the default `axios` export means interceptors registered by one feature apply to every request in the application — including calls to third-party APIs that must never receive your `Authorization` header. That is a credential leak to an external domain.

Failure mode: raw errors reaching components. Without an error-transformation interceptor, every component must know about `error.response?.data?.message` versus `error.request` versus `error.message`. Error handling becomes copy-paste, and the network-failure case is usually the one forgotten.

MOD-06.6 — Fetch in React

Use case. A typed client for an internal API, shared across the portal.

Failure mode: `fetch` resolves on 500. Only network failures reject. Code relying on `.catch` calls `.json()` on an HTML error page and throws an opaque parse error, hiding the real 503 from every alert and dashboard.

Failure mode: no timeout. `fetch` has no default timeout. A hung connection through a corporate proxy leaves a spinner rotating indefinitely with no error state and no recovery path.

Failure mode: hand-rolled server state. A team builds caching, deduplication, background refetch, stale-while-revalidate and retry on top of `fetch` — badly, over eighteen months, with no tests. They have written a worse RTK Query. **The adoption signal is concrete:** when you find yourself writing a cache key, you needed a server-state library three months ago.

MOD-06.7 — Generators and sagas

Use case. A tenant provisioning workflow: validate, reserve capacity, create, seed defaults, notify — cancellable at any step.

Failure mode: the leaked watcher. A saga started with `takeEvery` and never cancelled keeps a listener alive across route transitions. Navigate between twenty screens and twenty watchers are listening; one user action triggers twenty handlers, so the provisioning call fires twenty times. Cancellation on route change is the fix, and it is the reason sagas exist rather than an incidental hygiene rule.

Failure mode: the hanging generator. A generator awaiting a `take` that never arrives — because the awaited action was renamed — holds its closure and everything it captured, forever. Nothing errors; memory grows.

Failure mode: sagas adopted for CRUD. A team chooses redux-saga for simple fetch-and-store flows, adding a generator, a watcher, a worker and three action types per screen where `createAsyncThunk` is four lines. The complexity buys cancellation and orchestration nobody needs, and every new hire takes a week to become productive.

Part 3 — Architectural flowcharts and comparison tables

Diagram 1: the Axios 401 refresh flow with concurrent request queueing

```
sequenceDiagram
    participant W1 as Widget 1
    participant W2 as Widget 2
    participant W12 as Widget 12
    participant I as Response interceptor
    participant L as Refresh lock<br/>(module-scope promise)
    participant A as Auth service

    W1->>I: GET /metrics -> 401
    I->>L: isRefreshing? NO
```



```

I->>L: LOCK: store refreshPromise
I->>A: POST /auth/refresh
W2->>I: GET /quotas -> 401
I->>L: isRefreshing? YES
I->>L: enqueue (resolve, reject)
W12->>I: GET /jobs -> 401
I->>L: isRefreshing? YES
I->>L: enqueue (resolve, reject)
A-->>I: 200 { accessToken }
I->>L: flush queue with the NEW token
I->>L: UNLOCK: refreshPromise = null
I-->>W1: retry with new token -> 200
I-->>W2: retry with new token -> 200
I-->>W12: retry with new token -> 200

```

Five invariants make this correct, and dropping any one reproduces a Part 2 failure:

1. **One lock, module-scoped.** The in-flight refresh is a single promise held outside any request's closure. Twelve 401s see one promise, so `/auth/refresh` is called **exactly once**.
2. **Queue the waiters, do not retry them immediately.** Each 401 after the first parks a `(resolve, reject)` pair and awaits the shared promise.
3. **Mark every retry.** `config._retry = true` before re-firing. A second 401 on a retried request means the new token is genuinely rejected — log out rather than loop.
4. **Fail the queue on refresh failure.** If refresh returns 401 or 403, reject every queued waiter and redirect to login once. Leaving them pending hangs the UI forever.
5. **Never intercept the refresh call itself.** The refresh request must bypass this interceptor, or a failing refresh triggers a refresh, recursively.

Diagram 2: Context blast radius versus RTK selector subscriptions

```

flowchart TD
    subgraph CTX["Context: identity-based, whole-subtree"]
        A["Provider value object"] -->|"new identity"| B["React walks the tree"]
        B --> C["Consumer 1 re-renders"]
        B --> D["Consumer 2 re-renders"]
        B --> E["Consumer 2000 re-renders"]
        F["React.memo on a consumer"] -.->|"DOES NOT HELP:<br/>context bypasses props"| E
    end
    subgraph RTK["RTK: value-based, per-subscriber"]
        G["Store state changes"] --> H["Every subscriber's<br/>selector runs"]
        H --> I{"Object.is(prevResult,<br/>nextResult)?"}
        I -- "equal" --> J["SKIP re-render"]
        I -- "differs" --> K["Re-render only<br/>this component"]
        L["createSelector memo"] -.->|"same inputs =<br/>same reference"| I
    end
  
```

The asymmetry to internalize. Context propagates by **identity of the whole value**; a consumer reading only `theme` still re-renders when `searchQuery` changes. Redux propagates by **the value each subscriber selected**; a component selecting `state.tenants.entities[id].name` re-renders only when that name changes. This is why Context is dependency injection and Redux is state management, and why the fix for a Context blast radius is splitting providers rather than memoizing consumers.

Table 1: state management trade-off matrix

	Local state (<code>useState</code>)	Context API	Redux Toolkit	RTK Query / TanStack Query
Architectural intent	UI state owned by one component	Dependency injection for a subtree	Shared <i>client</i> state with an inspectable history	Server state: cache, sync, invalidate
Re-render blast radius	The component	Every consumer , on value identity change	Per-subscriber, by selected value	Per-hook, by query key and selected data

	Local state (<code>useState</code>)	Context API	Redux Toolkit	RTK Query / TanStack Query
Boilerplate	None	Low	Moderate (much reduced by RTK)	Low — and it deletes hand-written code
DevTools / time travel	No	No	Yes	Query devtools, not time travel
Normalization support	Manual	Manual	<code>createEntityAdapter</code>	Built in via tag invalidation
Async handling	Manual effects	Manual effects	<code>createAsyncThunk</code> or sagas	Built in: caching, dedup, refetch, retry
Ideal use case	Form drafts, toggles, hover, modal open	Theme, locale, auth identity, tenant id — values that rarely change	Cross-module client state: selections, wizard progress, optimistic edits	Anything that came from an API
Anti-pattern	Lifting everything to the root	Putting server data or fast-changing state in it	Putting form input state in it	Using it for pure client state

The decision path. Did it come from the server? → a server-state library. Is it client state needed by two or more distant modules? → Redux Toolkit. Is it a value that almost never changes but is needed deep in a tree? → Context. Otherwise → local state. Most enterprise state-management pain is one of these four answers applied to the wrong question.

Table 2: transport layer matrix

	Native fetch	Custom fetch wrapper	Axios
Bundle cost	0 kB — built in	~1–2 kB of your own code	~13 kB min+gzip
Interceptors	None	Whatever you write	First-class , request and response
HTTP errors	Resolves on 4xx/5xx — must check <code>res.ok</code>	Wrapper throws a typed error	Rejects automatically on non-2xx
JSON handling	Manual <code>.json()</code>	Automatic in the wrapper	Automatic both ways
Cancellation	<code>AbortController</code>	<code>AbortController</code>	<code>AbortController</code> (modern) or <code>CancelToken</code> (legacy)
Timeouts	None — build with <code>AbortSignal.timeout</code>	Built in	<code>timeout</code> option
Upload/download progress	Not available for uploads	Not available	Supported
Node support	Node 18+	Node 18+	All versions, plus adapter mocking
Testing ergonomics	Mock <code>global.fetch</code>	Mock your module	<code>axios-mock-adapter</code> , clean per-instance
Verdict	Fine for a handful of simple calls	Good default for a small, typed internal API surface	Correct for enterprise : the refresh queue, error transformation and per-instance isolation are worth 13 kB

The position this handbook takes. Use Axios for the authenticated first-party API where interceptors earn their keep. Use a thin typed `fetch` wrapper for public or unauthenticated endpoints where 13 kB and a shared interceptor chain buy nothing.

Never use the bare default `axios` export — always a configured instance per API surface.

Table 3: error classification and boundary hierarchy

Tier	Wraps	Catches	Fallback UI	Recovery path
Root shell	The whole app, outside the router	Render crashes in layout, router, providers, store wiring	Full-page apology, incident id, "Reload"	Hard reload. Last resort — reaching here means the shell is broken
Route layout	Each route's element, inside the router	Render crashes in a page, lazy chunk-load failures	In-layout error panel; navigation and shell stay usable	"Try again" resets the boundary via <code>resetKeys</code> ; navigation away also resets
Widget tile	Each dashboard tile, table or third-party embed	Render crashes in one widget, including third-party libraries	Placeholder inside the tile's own footprint	Per-tile retry; the other 39 tiles are unaffected

Error triage — what each class needs, and where it is handled:

Error class	Detected where	Correct response	Boundary involved
Render crash	Thrown during render	Fallback UI + telemetry + retry	The nearest boundary, by tier
Chunk-load failure	<code>React.lazy</code> import rejects	"A new version is available — reload"	Route layout (MOD-07.1 depends on this)
401 Unauthenticated	Response interceptor	Silent refresh; on failure, redirect to login with return-to	Not a boundary — transport layer
403 Forbidden	Response interceptor or route guard	Explicit "You do not have permission", naming who to ask	Route-level, rendered as a page
404 Not found	Response status or route match	Not-found page with navigation back to safety	Route-level

Error class	Detected where	Correct response	Boundary involved
500 / 503 Server	Response interceptor	Retry with backoff; then an error panel with a retry button	Route or widget tier, by scope
Network failure	<code>fetch rejects / Axios error.request</code>	"Check your connection", retry affordance	Whichever tier owns the request
Event-handler error	<code>try / catch</code> in the handler	Toast or inline message + telemetry	None — boundaries do not catch these

The rule this establishes. Boundaries catch *render* failures; the transport layer catches *network* failures. Confusing the two produces both a 401 that renders a crash page and a render crash that silently retries a request. Every fallback in every tier carries `role="alert"`, a real `<button>` for retry, and an incident id the user can quote to support.

Part 4 — Low-level deep dive and interactive coding lab

Strict TypeScript throughout. Work each debug challenge before reading its diagnosis.

MOD-06.1 — Context API

Mechanism. `createContext` produces a Provider and a consumption hook. When the Provider's `value` changes by `Object.is`, React notifies **every** consumer beneath it and re-renders them — bypassing `React.memo`, because context is not a prop. The only levers are: keep the value's identity stable, and split contexts so a volatile value does not sit beside a stable one.

```
// Bad — one context, inline value, volatile state beside stable identity
const AppContext = createContext<AppValue | undefined>(undefined);

export function AppProvider({ children }: { children: React.ReactNode }) {
  const [searchQuery, setSearchQuery] = useState(''); // changes per
  keystroke
  const [theme, setTheme] = useState<Theme>('light');
  return (
    <AppContext.Provider value={{ tenant, theme, setTheme, searchQuery,
  setSearchQuery }}>
      {children} /* 2,000 cells re-render on every keystroke */
    </AppContext.Provider>
  );
}
```



```

    );
  }

  // Production – split by volatility, memoized values, typed access hook
  interface TenantContextValue {
    readonly tenantId: string;
    readonly region: Region;
    readonly permissions: ReadonlySet<Permission>;
  }
  interface ThemeContextValue {
    readonly theme: Theme;
    readonly setTheme: (next: Theme) => void;
  }

  const TenantContext = createContext<TenantContextValue | undefined>
    (undefined);
  const ThemeContext = createContext<ThemeContextValue | undefined>(undefined);

  export function TenantProvider({
    tenant,
    children,
  }: {
    readonly tenant: Tenant;
    readonly children: React.ReactNode;
  }): React.ReactElement {
    // Stable unless the tenant genuinely changes (MOD-05.4)
    const value = useMemo<TenantContextValue>(
      () => ({
        tenantId: tenant.id,
        region: tenant.region,
        permissions: tenant.permissions,
      }),
      [tenant.id, tenant.region, tenant.permissions],
    );
    return <TenantContext.Provider value={value}>{children}</TenantContext.Provider>;
  }

  // Typed access hook: throws at the boundary instead of returning undefined
  export function useTenant(): TenantContextValue {
    const value = useContext(TenantContext);
    if (value === undefined) {
      throw new Error('useTenant must be used within a TenantProvider');
    }
    return value; // narrowed: callers never handle undefined
  }

```

Search query does not belong in Context at all — it is local state for the component that owns the search box, lifted no higher than the list it filters.

Splitting state from dispatch. When a context must hold both, use two providers: a value context that changes and a dispatch context that never does. Components that only dispatch then never re-render on value changes.

Debug challenge DBG-06.1

```
const SettingsContext = createContext<Settings>({ pageSize: 25, dense: false
});

function SettingsProvider({ children }: { children: React.ReactNode }) {
  const [settings, setSettings] = useState<Settings>({ pageSize: 25, dense:
false });
  const update = (patch: Partial<Settings>) => setSettings({ ...settings,
...patch });
  return (
    <SettingsContext.Provider value={{ ...settings, update }}>
      {children}
    </SettingsContext.Provider>
  );
}
```

► Reveal diagnosis

Three defects. First, the value is a fresh object literal on **every render of the provider**, including renders caused by an unrelated parent, so every consumer re-renders even when settings have not changed — wrap it in `useMemo`. Second, `update` is a new function each render and closes over `settings`, so two rapid calls collide on one snapshot; use `useCallback` with a functional updater. Third, the default value `{ pageSize: 25, dense: false }` is a silent trap: a consumer rendered outside the provider gets working defaults and never errors, so a missing provider is discovered in production rather than at the first render. Default to `undefined` and throw in the access hook.

MOD-06.2 — Error boundaries

Mechanism. Only a class component can be an error boundary, because the hook equivalents do not exist. `static getDerivedStateFromError(error)` runs during the render phase and returns the state that renders the fallback — it must be **pure**, so no logging here. `componentDidCatch(error, errorInfo)` runs in the commit phase and is where telemetry belongs, because side effects are allowed there. Boundaries catch errors in the render phase of their **children only**: not in event handlers, not in async code, not in themselves.

```
// Bad – no reset, no telemetry, no triage, side effect in a pure method
class ErrorBoundary extends React.Component<Props, { hasError: boolean }> {
  static getDerivedStateFromError() {
    logToServer(); // side effect in a PURE method: runs
twice
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) return <p>Something went wrong</p>; // no retry
    return this.props.children;
  }
}

// Production – typed, resettable, telemetry in the right lifecycle,
accessible
interface ErrorBoundaryProps {
  readonly children: React.ReactNode;
  readonly fallback: (state: FallbackState) => React.ReactNode;
  readonly onError?: (error: Error, info: React.ErrorInfo, incidentId:
string) => void;
  /** Changing any member resets the boundary – e.g. [location.pathname,
tenantId] */
  readonly resetKeys?: readonly unknown[];
  readonly tier: 'root' | 'route' | 'widget';
}

export interface FallbackState {
  readonly error: Error;
  readonly incidentId: string;
  readonly reset: () => void;
}

interface ErrorBoundaryState {
  readonly error: Error | null;
  readonly incidentId: string | null;
}

export class ErrorBoundary extends React.Component<ErrorBoundaryProps,
ErrorBoundaryState> {
  override state: ErrorBoundaryState = { error: null, incidentId: null };

  static getDerivedStateFromError(error: Error): ErrorBoundaryState {
    // PURE: derive state only. Telemetry happens in componentDidCatch.
    return { error, incidentId: crypto.randomUUID() };
  }
}
```

```

    override componentDidCatch(error: Error, info: React.ErrorInfo): void {
      if (this.state.incidentId !== null) {
        this.props.onError?.(error, info, this.state.incidentId);
      }
    }

    override componentDidUpdate(prev: ErrorBoundaryProps): void {
      const { resetKeys } = this.props;
      if (
        this.state.error !== null &&
        resetKeys !== undefined &&
        !shallowEqualArrays(prev.resetKeys, resetKeys)
      ) {
        this.reset();    // navigating away clears the error automatically
      }
    }

    private readonly reset = (): void => {
      this.setState({ error: null, incidentId: null });
    };

    override render(): React.ReactNode {
      const { error, incidentId } = this.state;
      if (error !== null && incidentId !== null) {
        return this.props.fallback({ error, incidentId, reset: this.reset });
      }
      return this.props.children;
    }
  }
}

```

Note the `fallback` render-prop signature. It hands the fallback a `reset` function, which is what makes recovery possible — a fallback that cannot reset is a dead end, and "reload the page" is not a recovery strategy for a single broken tile.

Debug challenge DBG-06.2

```

<ErrorBoundary tier="widget" fallback={() => <p>Chart unavailable</p>>
  <TelemetryChart
    onPointClick={point => {
      throw new Error(`Bad point: ${point.id}`);
    }}
  />
</ErrorBoundary>

```

► Reveal diagnosis

The boundary never fires. `onPointClick` runs in an event handler, not during render, so the throw escapes to the window as an unhandled error and the chart keeps rendering as if nothing happened. Error boundaries catch render-phase errors only. Handle it in the handler with `try / catch` and either set error state (which *will* surface on the next render, where a boundary can see it) or show an inline message plus telemetry. The same applies to `setTimeout` callbacks, promise rejections and `useEffect` bodies — none reach a boundary.

MOD-06.3 — Redux architecture

Mechanism. One store holds the whole client state tree. State changes only by dispatching a plain-object action. A reducer is `(state, action) => newState` — pure, synchronous, no I/O, no randomness, no mutation. Middleware sits between `dispatch` and the reducer as a curried chain, `store => next => action => result`, which is MOD-02.2 applied to a pipeline.

```
// The middleware signature IS currying from MOD-02.2
const logger: Middleware<{}, RootState> = store => next => action => {
  console.groupCollapsed(String((action as AnyAction).type));
  console.log('prev', store.getState());
  const result = next(action);      // MUST return, or the chain dies here
  console.log('next', store.getState());
  console.groupEnd();
  return result;
};
```

Why Redux exists. Not to share state — Context does that. Redux buys four properties that matter at scale: every change has a **named cause** (the action type) so the log reads as a narrative; state transitions are **replayable**, which is what time-travel and bug reports built from action logs depend on; cross-cutting behaviour lives in **one pipeline** rather than scattered through components; and subscribers re-render by **selected value** rather than by tree position. If none of those four matter for a given piece of state, Redux is the wrong tool for it.

Debug challenge DBG-06.3

```
function jobsReducer(state: JobsState = initialState, action: AnyAction):
JobsState {
  switch (action.type) {
    case 'jobs/received':
      state.items = action.payload;
      state.lastFetched = Date.now();
      return state;
    default:
      return state;
  }
}
```

► Reveal diagnosis

Three violations in four lines, and this is a plain reducer with no Immer to save it. It **mutates** `state` and returns the same reference, so `Object.is` reports no change and no subscriber re-renders — the UI never updates. It calls `Date.now()`, making the reducer **impure**: replaying the same action log produces different state each time, so DevTools time-travel silently lies. And `action` is typed `AnyAction`, so `action.payload` is `any` — a zero-`any` violation that removes every guarantee at the boundary where data enters the store. Return a new object, pass the timestamp **in the action payload** (computed by the action creator or thunk, where side effects are legal), and type the action.

MOD-06.4 — Redux Toolkit

Mechanism. `configureStore` wires the store with sensible middleware and DevTools. `createSlice` generates action creators and types from reducer names. Inside a slice, Immer hands your reducer a **draft proxy**: assignments to the draft are recorded and applied to produce a new immutable state. The mutation is an illusion, and it only holds **inside** a `createSlice` reducer or `createReducer`. `createEntityAdapter` supplies the normalized `{ ids, entities }` shape and CRUD reducers; `createSelector` memoizes projections.

```
// Bad – duplicated nested data, allocating selector, untyped thunk
interface BadState {
  permissions: { user: User; perms: string[] }[]; // user duplicated
  billing: { user: User; plan: string }[]; // and again
}
const selectUsers = (s: RootState) =>
  s.permissions.map(p => p.user); // NEW ARRAY every call

// Production – normalized, typed, memoized
export interface TenantUser {
  readonly id: string;
```

```

    readonly name: string;
    readonly email: string;
    readonly roleIds: readonly string[];
    readonly active: boolean;
  }

  const usersAdapter = createEntityAdapter<TenantUser>({
    sortComparer: (a, b) => a.name.localeCompare(b.name),
  });

  // Derived by reduce under the hood: { ids: string[], entities:
  Record<string, T> }
  const initialState = usersAdapter.getInitialState<{
    readonly status: 'idle' | 'loading' | 'succeeded' | 'failed';
    readonly error: string | null;
  }>({ status: 'idle', error: null });

  export const usersSlice = createSlice({
    name: 'users',
    initialState,
    reducers: {
      userUpdated: usersAdapter.updateOne,    // draft mutation handled by Immer
      userRemoved: usersAdapter.removeOne,
    },
  });

  // Memoized projection: same inputs -> same reference -> no re-render
  export const selectActiveUserCount = createSelector(
    [usersAdapter.getSelectors<RootState>(s => s.users).selectAll],
    users => users.filter(u => u.active).length,    // recomputes only when
    users change
  );

```

RootState and **AppDispatch** are inferred, never written by hand:

```

export const store = configureStore({ reducer: { users: usersSlice.reducer }
});
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;

```

Hand-writing **RootState** guarantees it drifts from the real store the first time a slice is added.

DevTools hygiene. Action types read as `domain/event` past tense (`users/userUpdated`), not as commands. Non-serializable values (class instances, `Date`, functions) never enter the store — RTK's serializability middleware catches them in development and it should stay on. DevTools are disabled in production builds.

Debug challenge DBG-06.4

```
export const fetchUsers = createAsyncThunk(
  'users/fetch',
  async (tenantId: string, { getState }) => {
    const state = getState() as RootState;
    state.users.status = 'loading';          // (1)
    const res = await api.get(`/tenants/${tenantId}/users`);
    return res.data;
  },
);

const selectSorted = (s: RootState) =>
  selectAllUsers(s).sort((a, b) => a.name.localeCompare(b.name)); // (2)
```

► Reveal diagnosis

Line (1) mutates the **real** store state, not an Immer draft — thunks run outside `createSlice`, so there is no proxy. The store reference does not change, so nothing re-renders, and the action log no longer describes the state: this is exactly the non-deterministic time-travel defect from Part 2. Status belongs in `extraReducers` responding to `fetchUsers.pending`. The `as RootState` cast is a second violation; type the thunk with `createAsyncThunk<Returned, Arg, { state: RootState }>` so `getState()` is typed.

Line (2) has two problems. `sort` **mutates** the array — and `selectAllUsers` returns the adapter's memoized array, so this mutates memoized state in place (MOD-01.6). It also returns a new array every call, so `useSelector`'s reference check always fails and the component re-renders on every dispatch from any slice. Write `createSelector([selectAllUsers], users => [...users].sort(...))`.

MOD-06.5 — Axios in the enterprise

Mechanism. `axios.create(config)` produces an **isolated instance** with its own interceptor chain, base URL and defaults. Interceptors are registered functions that wrap a request before it leaves or a response before it resolves — the decorator pattern from MOD-02.2. Response interceptors receive two callbacks: one for success, one for errors, and the error callback is where the 401 refresh lives.


```

// Bad – global instance, refresh stampede, infinite retry loop
axios.interceptors.response.use(undefined, async error => {
  if (error.response?.status === 401) {
    const { data } = await axios.post('/auth/refresh'); // every 401
    refreshes
    setToken(data.accessToken);
    return axios(error.config); // no _retry flag: loops forever on
  failure
  }
  return Promise.reject(error);
});

// Production – isolated instance, single-flight refresh, queued waiters
interface QueuedRequest {
  readonly resolve: (token: string) => void;
  readonly reject: (error: unknown) => void;
}

let refreshPromise: Promise<string> | null = null; // module-scoped LOCK
let queue: QueuedRequest[] = [];

export const apiClient: AxiosInstance = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 15_000,
  headers: { 'Content-Type': 'application/json' },
});

// A SEPARATE instance for refresh: it must never hit this interceptor
const authClient: AxiosInstance = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 10_000,
});

apiClient.interceptors.request.use(config => {
  const token = tokenStore.getAccessToken();
  if (token !== null) config.headers.Authorization = `Bearer ${token}`;
  config.headers['X-Correlation-Id'] = getCorrelationId();
  return config;
});

apiClient.interceptors.response.use(
  response => response,
  async (error: unknown): Promise<AxiosResponse> => {
    if (!axios.isAxiosError(error) || error.config === undefined) {
      return Promise.reject(toApiError(error));
    }
  }

```

```

const original = error.config as RetryableConfig;

// Not a 401, or we already retried this one: give up, do not loop
if (error.response?.status !== 401 || original._retry === true) {
  return Promise.reject(toApiError(error));
}
original._retry = true;

// Refresh already in flight: queue and wait for the SAME promise
if (refreshPromise !== null) {
  return new Promise<AxiosResponse>((resolve, reject) => {
    queue.push({
      resolve: (token: string) => {
        original.headers.Authorization = `Bearer ${token}`;
        resolve(apiClient(original));
      },
      reject,
    });
  });
}

// We are the first 401: acquire the lock
refreshPromise = authClient
  .post<{ accessToken: string }>('/auth/refresh')
  .then(res => {
    tokenStore.setAccessToken(res.data.accessToken);
    return res.data.accessToken;
  });

try {
  const token = await refreshPromise;
  queue.forEach(q => q.resolve(token)); // flush the waiters
  queue = [];
  original.headers.Authorization = `Bearer ${token}`;
  return await apiClient(original);
} catch (refreshError) {
  queue.forEach(q => q.reject(refreshError)); // never leave them
  pending
    queue = [];
    tokenStore.clear();
    redirectToLogin();
    return await Promise.reject(toApiError(refreshError));
} finally {

```

```

        refreshPromise = null;                                // release the lock, always
    }
  },
);

```

The `finally` is not optional. If an unexpected throw skips the unlock, `refreshPromise` stays non-null forever and every subsequent 401 queues against a settled promise that will never flush — the tab hangs silently.

Debug challenge DBG-06.5

```

client.interceptors.response.use(
  response => response,
  async error => {
    if (error.response?.status === 401 && !isRefreshing) {
      isRefreshing = true;
      const token = await refresh();
      isRefreshing = false;
      return client(error.config);
    }
    return Promise.reject(error);
  },
);

```

► Reveal diagnosis

The boolean flag prevents a *second refresh call* but does nothing for the other eleven requests: they fall through to `Promise.reject(error)` and **fail with 401** even though a valid token arrives milliseconds later. Eleven widgets render error states on a perfectly recoverable condition. A boolean is not a queue — you need somewhere for the waiters to park, which is what the shared `refreshPromise` plus the `queue` array provide. Two further defects: `isRefreshing = false` is not in a `finally`, so a thrown refresh leaves the flag stuck `true` and no future 401 is ever handled; and there is no `_retry` flag, so a refresh that returns a still-invalid token loops indefinitely.

MOD-06.6 — Fetch in React

Mechanism. `fetch` is the platform primitive: zero bytes, resolves on any HTTP status, no timeout, cancellation through `AbortController`. A typed wrapper adds the three things it lacks — status checking, timeouts, and a typed error union — without the 13 kB.

```

// Bad — resolves on 500, no timeout, no abort, response typed as any
async function getUsers(tenantId: string) {
  const res = await fetch(`/api/tenants/${tenantId}/users`);
  return res.json();           // any; may parse an HTML error page
}

```

```

}

// Production — typed wrapper with status checks, timeout and abort
export type ApiResult<T> =
  | { readonly ok: true; readonly data: T }
  | { readonly ok: false; readonly error: ApiError };

export type ApiError =
  | { readonly kind: 'http'; readonly status: number; readonly message:
string }
  | { readonly kind: 'network'; readonly message: string }
  | { readonly kind: 'timeout'; readonly message: string }
  | { readonly kind: 'parse'; readonly message: string };

export async function apiGet<T>(
  path: string,
  init: { readonly signal?: AbortSignal; readonly timeoutMs?: number } = {},
): Promise<ApiResult<T>> {
  const { signal, timeoutMs = 15_000 } = init;
  // Compose the caller's signal with our timeout
  const timeout = AbortSignal.timeout(timeoutMs);
  const composed = signal ? AbortSignal.any([signal, timeout]) : timeout;

  try {
    const res = await fetch(path, {
      signal: composed,
      headers: { Accept: 'application/json' },
    });
    if (!res.ok) {
      return { ok: false, error: { kind: 'http', status: res.status, message:
res.statusText } };
    }
    const data = (await res.json()) as T; // the one boundary a schema
belongs at
    return { ok: true, data };
  } catch (error) {
    if (error instanceof Error && error.name === 'TimeoutError') {
      return { ok: false, error: { kind: 'timeout', message: `Timed out after
${timeoutMs}ms` } };
    }
    if (error instanceof Error && error.name === 'AbortError') {
      throw error; // deliberate cancellation: let the caller ignore it
    }
    if (error instanceof SyntaxError) {
      return { ok: false, error: { kind: 'parse', message: 'Malformed
response body' } };
    }
  }
}

```

```

    }
    return { ok: false, error: { kind: 'network', message: 'Network request
failed' } };
  }
}

```

The `ApiResponse<T>` union forces every call site to handle failure — there is no way to read `.data` without narrowing on `ok` first. `AbortError` is rethrown rather than returned as a result, because a cancellation is not an outcome the UI should render.

When to adopt a server-state library. Any two of these mean the decision has already been made: you are writing cache keys; you have a `lastFetched` timestamp in Redux; two components fetch the same endpoint independently; you have implemented refetch-on-focus; or you are hand-writing optimistic updates with rollback. RTK Query is the natural choice when Redux is already present, TanStack Query when it is not.

Debug challenge DBG-06.6

```

useEffect(() => {
  const controller = new AbortController();
  apiGet<User[]>('/api/users', { signal: controller.signal })
    .then(result => {
      if (result.ok) setUsers(result.data);
      else setError(result.error.message);
    });
  return () => controller.abort();
}, []);

```

► Reveal diagnosis

The abort path throws rather than resolving, and there is no `.catch`, so aborting on unmount produces an **unhandled promise rejection** on every navigation — noisy in the console and, in some monitoring setups, reported as an error spike. Add `.catch(err => { if (err.name !== 'AbortError') setError(...); })`. Note the design tension this exposes: returning cancellation as a result value would force every call site to handle a case it does not care about, while throwing forces every call site to add a catch. The wrapper chooses throwing because a missing catch is loud, whereas an unhandled result variant is silent.

MOD-06.7 — Generators and sagas

Mechanism. `function*` returns a generator that conforms to the **iterator protocol**: calling `.next()` runs until the next `yield` and returns `{ value, done }`. Execution is suspended, not blocked, and the caller decides when to resume. Communication is **two-**

way: `yield` sends a value out, and the argument passed to `.next(v)` becomes the result of that `yield` expression inside the generator. `yield*` delegates to another iterable.

```
// Lazy sequence: infinite, but only computes what is consumed
function* pageNumbers(): Generator<number, never, unknown> {
  let page = 1;
  while (true) yield page++;
}

// Two-way communication: the caller injects the result back in
function* provisioning(): Generator<Effect, string, unknown> {
  const valid = (yield { type: 'CALL', fn: validate }) as boolean; //
  resumed value
  if (!valid) return 'invalid';
  const id = (yield { type: 'CALL', fn: createTenant }) as string;
  yield* seedDefaults(id);      // delegate to another generator
  return id;
}
```

The relationship to `async/await`. `async / await` is generators plus a scheduler: the transpiled output of an `async` function is a generator driven by a runner that calls `.next()` each time a promise settles. That is the whole connection, and it closes the arc opened in MOD-02.4.

Why sagas exist. A saga yields plain **effect descriptions** — `call`, `put`, `take`, `fork`, `cancel`, `race` — rather than performing the work. The middleware interprets them. Two consequences follow: the saga is trivially testable because you assert on the yielded objects with no mocking at all, and the middleware can **cancel** a saga mid-flight, which `async / await` cannot do once started.

```
function* watchProvisioning(): Generator {
  // takeLatest cancels any in-flight run when a new request arrives
  yield takeLatest('tenants/provisionRequested', provisionSaga);
}

function* provisionSaga(action: PayloadAction<ProvisionRequest>): Generator {
  try {
    yield put(provisionStarted());
    const { result, timeout } = (yield race({
      result: call(api.provisionTenant, action.payload),
      timeout: delay(30_000),
    })) as { result?: Tenant; timeout?: true };

    if (timeout !== undefined) {
      yield put(provisionFailed('Provisioning timed out'));
    }
  }
}
```

```

    return;
  }
  yield put(provisionSucceeded(result as Tenant));
} finally {
  // Runs on cancellation too: this is the leak fix from Part 2
  if (yield cancelled()) {
    yield put(provisionAborted());
  }
}
}
}

```

Debug challenge DBG-06.7

```

function* watchJobs(): Generator {
  yield takeEvery('jobs/subscribe', function* (action) {
    const channel = yield call(createJobChannel, action.payload.tenantId);
    while (true) {
      const event = yield take(channel);
      yield put(jobEventReceived(event));
    }
  });
}

```

► Reveal diagnosis

A leaked watcher per navigation. `takeEvery` starts a **new** worker for every `jobs/subscribe` action and never cancels the previous one, and each worker sits in an infinite `while (true)` loop holding an open channel. Navigate between twenty tenant screens and twenty channels are open, twenty workers push into the store, and the same job event dispatches twenty times. There is also no `finally` block, so even an external `cancel` would leave the channel open. Use `takeLatest` so a new subscription cancels the previous run, and close the channel in a `finally` guarded by `yield cancelled()`. The general rule matches MOD-05.2's: **anything a saga opens, the saga must close on cancellation**.

Coding Lab LAB-06 — EnterpriseDataOrchestrationLayer

Brief. Three production modules that together form the data layer of a multi-tenant console: a resettable error boundary with telemetry, a normalized RTK slice with memoized selectors, and an Axios client with a thread-safe refresh queue.

1. `ErrorBoundary.tsx` — accessible fallbacks by tier

The boundary class is the one from MOD-06.2 above. What follows is the tiered fallback layer that makes it usable, and the accessibility contract it satisfies.

```
// fallbacks.tsx
import type { FallbackState } from './ErrorBoundary';

export function WidgetFallback({ error, incidentId, reset }: FallbackState) {
  return (
    <div role="alert">
      <p>This tile could not be displayed.</p>
      <p>Incident {incidentId}</p>
      /* A real button: keyboard-operable, focusable, announced as a button
*/
      <button type="button" onClick={reset}>
        Retry loading this tile
      </button>
    </div>
  );
}

export function RouteFallback({ error, incidentId, reset }: FallbackState) {
  const isChunkError = /chunk|dynamically imported
module/i.test(error.message);
  return (
    <div role="alert">
      <h1>{isChunkError ? 'A new version is available' : 'This page could not
load'}</h1>
      <p>
        {isChunkError
          ? 'Reload to get the latest version of the console.'
          : 'The rest of the console is still available from the
navigation.'}
      </p>
      <p>Incident {incidentId}</p>
      <button type="button" onClick={isChunkError ? () => location.reload() :
reset}>
        {isChunkError ? 'Reload the console' : 'Try again'}
      </button>
    </div>
  );
}

// Usage: resetKeys clear the error automatically on navigation or tenant
switch
<ErrorBoundary
  tier="route"
  fallback={state => <RouteFallback {...state} />}
  resetKeys={[location.pathname, tenantId]}
  onError={(error, info, incidentId) =>
```



```

    telemetry.captureException(error, {
      incidentId,
      componentStack: info.componentStack,
      tier: 'route',
      tenantId,
    })
  }
}
>
<Outlet />
</ErrorBoundary>

```

The chunk-error branch is the MOD-07.1 dependency made concrete: a failed dynamic import is the one render error where retrying in place cannot work, because the old chunk hash no longer exists on the CDN.

2. **tenantSlice.ts** — normalized entities and memoized projections

```

// tenantSlice.ts
import {
  createAsyncThunk,
  createEntityAdapter,
  createSelector,
  createSlice,
  type PayloadAction,
} from '@reduxjs/toolkit';
import { apiClient } from '../apiClient';
import type { RootState } from '../store';

// ----- Domain -----

export interface TenantUser {
  readonly id: string;
  readonly name: string;
  readonly email: string;
  readonly roleIds: readonly string[];
  readonly active: boolean;
}

export interface Role {
  readonly id: string;
  readonly name: string;
  readonly permissionIds: readonly string[];
}

export interface Permission {
  readonly id: string;
  readonly key: string;
  readonly description: string;
}

```

```

}

export interface TenantPayload {
  readonly users: readonly TenantUser[];
  readonly roles: readonly Role[];
  readonly permissions: readonly Permission[];
}

export type LoadStatus = 'idle' | 'loading' | 'succeeded' | 'failed';

// ----- Adapters: the { ids, entities } shape from MOD-01.8 -----

const usersAdapter = createEntityAdapter<TenantUser>({
  sortComparer: (a, b) => a.name.localeCompare(b.name),
});
const rolesAdapter = createEntityAdapter<Role>();
const permissionsAdapter = createEntityAdapter<Permission>();

interface TenantSliceState {
  readonly users: ReturnType<typeof usersAdapter.getInitialState>;
  readonly roles: ReturnType<typeof rolesAdapter.getInitialState>;
  readonly permissions: ReturnType<typeof
permissionsAdapter.getInitialState>;
  readonly status: LoadStatus;
  readonly error: string | null;
  readonly tenantId: string | null;
}

const initialState: TenantSliceState = {
  users: usersAdapter.getInitialState(),
  roles: rolesAdapter.getInitialState(),
  permissions: permissionsAdapter.getInitialState(),
  status: 'idle',
  error: null,
  tenantId: null,
};

// ----- Thunk: fully typed, no casts -----

export const fetchTenantGraph = createAsyncThunk<
  TenantPayload, // Returned
  string, // Arg: tenantId
  { state: RootState; rejectValue: string } // ThunkApiConfig
>('tenant/fetchGraph', async (tenantId, { rejectWithValue, signal }) => {
  try {
    const res = await apiClient.get<TenantPayload>

```

```

(`/tenants/${tenantId}/graph`, {
  signal, // RTK aborts on dispatch
cancel
});
return res.data;
} catch (error) {
  return rejectWithValue(toApiError(error).message);
}
});

// ----- Slice -----

export const tenantSlice = createSlice({
  name: 'tenant',
  initialState,
  reducers: {
    // Immer draft: this LOOKS mutable and is not. Only valid inside a slice.
    userUpdated(state, action: PayloadAction<{ id: string; changes:
Partial<TenantUser> }>) {
      usersAdapter.updateOne(state.users, action.payload);
    },
    userRemoved(state, action: PayloadAction<string>) {
      usersAdapter.removeOne(state.users, action.payload);
    },
    userUpserted(state, action: PayloadAction<TenantUser>) {
      usersAdapter.upsertOne(state.users, action.payload);
    },
    tenantCleared() {
      return initialState; // returning replaces the
draft
    },
  },
  extraReducers: builder => {
    builder
      .addCase(fetchTenantGraph.pending, (state, action) => {
        state.status = 'loading';
        state.error = null;
        state.tenantId = action.meta.arg;
      })
      .addCase(fetchTenantGraph.fulfilled, (state, action) => {
        state.status = 'succeeded';
        // setAll REPLACES: no duplicate user objects across modules
        usersAdapter.setAll(state.users, action.payload.users);
        rolesAdapter.setAll(state.roles, action.payload.roles);
        permissionsAdapter.setAll(state.permissions,
action.payload.permissions);

```

```

    })
    .addCase(fetchTenantGraph.rejected, (state, action) => {
      state.status = 'failed';
      state.error = action.payload ?? 'Failed to load tenant data';
    });
  },
});

export const { userUpdated, userRemoved, userUpserted, tenantCleared } =
  tenantSlice.actions;

// ----- Selectors: memoized, never allocating in the component -----
--

export const usersSelectors = usersAdapter.getSelectors<RootState>(s =>
s.tenant.users);
export const rolesSelectors = rolesAdapter.getSelectors<RootState>(s =>
s.tenant.roles);
export const permissionsSelectors =
permissionsAdapter.getSelectors<RootState>(
  s => s.tenant.permissions,
);

export const selectTenantStatus = (s: RootState): LoadStatus =>
s.tenant.status;

export const selectActiveUsers = createSelector(
  [usersSelectors.selectAll],
  users => users.filter(u => u.active),           // stable reference between
changes
);

export const selectActiveUserCount = createSelector(
  [selectActiveUsers],
  users => users.length,                         // a number: never re-
renders spuriously
);

/** Resolves a user's effective permissions across their roles. 0(roles +
perms). */
export const selectEffectivePermissions = createSelector(
  [
    usersSelectors.selectEntities,
    rolesSelectors.selectEntities,
    permissionsSelectors.selectEntities,
    (_state: RootState, userId: string) => userId,

```

```

],
(userEntities, roleEntities, permissionEntities, userId): readonly
Permission[] => {
  const user = userEntities[userId];
  if (user === undefined) return [];
  const keys = new Set<string>();
  for (const roleId of user.roleIds) {
    const role = roleEntities[roleId];
    if (role === undefined) continue;
    for (const permissionId of role.permissionIds) keys.add(permissionId);
  }
  // reduce into a deduplicated list – MOD-01.8, one pass
  return [...keys]
    .map(id => permissionEntities[id])
    .filter((p): p is Permission => p !== undefined);
},
);

```

Normalization is the whole point: the user object exists **once**. The permissions module and the billing module both read `usersSelectors.selectById(state, id)` and see the same record, so a name change cannot produce two truths. The parameterized selector takes `userId` as an input selector rather than closing over it, which keeps it usable from any component.

3. `apiClient.ts` — thread-safe 401 refresh with request queueing

```

// apiClient.ts
import axios, {
  type AxiosError,
  type AxiosInstance,
  type AxiosResponse,
  type InternalAxiosRequestConfig,
} from 'axios';
import { tokenStore } from './tokenStore';

// ----- Typed error contract -----

export type ApiError =
  | { readonly kind: 'http'; readonly status: number; readonly message:
string; readonly correlationId?: string }
  | { readonly kind: 'network'; readonly message: string }
  | { readonly kind: 'timeout'; readonly message: string }
  | { readonly kind: 'cancelled'; readonly message: string };

interface RetryableConfig extends InternalAxiosRequestConfig {
  _retry?: boolean;
}

```

```

}

interface QueuedWaiter {
  readonly resolve: (token: string) => void;
  readonly reject: (reason: unknown) => void;
}

// ----- Module-scoped lock. ONE per module, never per request. -----
--

let refreshPromise: Promise<string> | null = null;
let waiters: QueuedWaiter[] = [];

/** Test-only reset so state does not leak between specs. */
export function __resetAuthState(): void {
  refreshPromise = null;
  waiters = [];
}

// ----- Instances: isolated, never the bare `axios` export -----

export const apiClient: AxiosInstance = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 15_000,
  headers: { 'Content-Type': 'application/json' },
});

/** Separate instance: MUST NOT pass through the 401 interceptor. */
export const authClient: AxiosInstance = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 10_000,
});

// ----- Request interceptor -----

apiClient.interceptors.request.use(config => {
  const token = tokenStore.getAccessToken();
  if (token !== null) {
    config.headers.set('Authorization', `Bearer ${token}`);
  }
  config.headers.set('X-Correlation-Id', tokenStore.getCorrelationId());
  return config;
});

// ----- Response interceptor: the refresh queue -----

```

```

apiClient.interceptors.response.use(
  response => response,
  async (error: unknown): Promise<AxiosResponse> => {
    if (!axios.isAxiosError(error)) {
      return Promise.reject<never>(toApiError(error));
    }
    const axiosError = error as AxiosError;
    const original = axiosError.config as RetryableConfig | undefined;

    const isAuthFailure = axiosError.response?.status === 401;
    if (original === undefined || !isAuthFailure || original._retry === true)
    {
      // Not recoverable here: a retried request that 401s again means the
      // refreshed token is genuinely rejected. Do NOT loop.
      if (original?._retry === true) {
        tokenStore.clear();
        redirectToLogin();
      }
      return Promise.reject<never>(toApiError(axiosError));
    }

    original._retry = true;

    // A refresh is already in flight: park and await the SAME promise.
    if (refreshPromise !== null) {
      return new Promise<AxiosResponse>((resolve, reject) => {
        waiters.push({
          resolve: (token: string) => {
            original.headers.set('Authorization', `Bearer ${token}`);
            resolve(apiClient(original));
          },
          reject,
        });
      });
    }

    // First 401 wins the lock and performs the single refresh.
    refreshPromise = authClient
      .post<{ accessToken: string }>('/auth/refresh', {
        refreshToken: tokenStore.getRefreshToken(),
      })
      .then(res => {
        tokenStore.setAccessToken(res.data.accessToken);
        return res.data.accessToken;
      });
  }
);

```

```

    try {
      const token = await refreshPromise;
      const parked = waiters;
      waiters = [];
      parked.forEach(w => w.resolve(token));          // flush every waiter
      original.headers.set('Authorization', `Bearer ${token}`);
      return await apiClient(original);
    } catch (refreshError) {
      const parked = waiters;
      waiters = [];
      parked.forEach(w => w.reject(refreshError));    // never leave them
    }
    pending
      tokenStore.clear();
      redirectToLogin();
      return await Promise.reject<never>(toApiError(refreshError));
    } finally {
      refreshPromise = null;                          // release the lock,
    }
  },
);

// ----- Error transformation: components never see raw Axios -----

export function toApiError(error: unknown): ApiError {
  if (axios.isCancel(error)) {
    return { kind: 'cancelled', message: 'Request cancelled' };
  }
  if (axios.isAxiosError(error)) {
    if (error.code === 'ECONNABORTED') {
      return { kind: 'timeout', message: 'The request timed out' };
    }
    if (error.response !== undefined) {
      const body = error.response.data as { message?: string } | undefined;
      return {
        kind: 'http',
        status: error.response.status,
        message: body?.message ?? error.response.statusText,
        correlationId: error.response.headers['x-correlation-id'] as string |
        undefined,
      };
    }
    return { kind: 'network', message: 'Unable to reach the server' };
  }
  return { kind: 'network', message: 'An unexpected error occurred' };
}

```


The three lines that carry the whole design. `refreshPromise` is module-scoped, so twelve concurrent 401s observe one in-flight refresh. `original._retry = true` makes a second 401 on the same request terminal rather than recursive. And `finally { refreshPromise = null }` guarantees the lock is released even on an unexpected throw — without it a single edge case hangs every future request in the tab.

4. Jest + React Testing Library suite

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import MockAdapter from 'axios-mock-adapter';
import { configureStore } from '@reduxjs/toolkit';
import { ErrorBoundary } from './ErrorBoundary';
import { WidgetFallback } from './fallbacks';
import {
  tenantSlice,
  fetchTenantGraph,
  userUpdated,
  userRemoved,
  usersSelectors,
  selectActiveUsers,
  selectActiveUserCount,
  selectEffectivePermissions,
  type TenantUser,
} from './tenantSlice';
import { apiClient, authClient, __resetAuthState, toApiError } from
'./apiClient';

// ===== 1. ErrorBoundary =====

function Boom({ shouldThrow }: { readonly shouldThrow: boolean }):
React.ReactElement {
  if (shouldThrow) throw new Error('Chart exploded');
  return <p>Chart is fine</p>;
}

describe('ErrorBoundary', () => {
  // React logs caught errors; silence only that noise
  beforeEach(() => jest.spyOn(console, 'error').mockImplementation(() =>
{}));
  afterEach(() => jest.restoreAllMocks());

  it('renders children when nothing throws', () => {
    render(
      <ErrorBoundary tier="widget" fallback={s => <WidgetFallback {...s} />>
        <Boom shouldThrow={false} />
    )
  }
}
```

```

    </ErrorBoundary>,
  );
  expect(screen.getByText('Chart is fine')).toBeInTheDocument();
  expect(screen.queryByRole('alert')).not.toBeInTheDocument();
});

it('renders an ACCESSIBLE fallback when a child throws', () => {
  render(
    <ErrorBoundary tier="widget" fallback={s => <WidgetFallback {...s} />}>
      <Boom shouldThrow />
    </ErrorBoundary>,
  );
  expect(screen.getByRole('alert')).toBeInTheDocument();
  expect(
    screen.getByRole('button', { name: /retry loading this tile/i }),
  ).toBeInTheDocument();
});

it('emits telemetry ONCE with an incident id and component stack', () => {
  const onError = jest.fn();
  render(
    <ErrorBoundary tier="widget" fallback={s => <WidgetFallback {...s} />}
onError={onError}>
      <Boom shouldThrow />
    </ErrorBoundary>,
  );
  expect(onError).toHaveBeenCalledTimes(1);
  const [error, info, incidentId] = onError.mock.calls[0];
  expect(error).toBeInstanceOf(Error);
  expect(info.componentStack).toBeTruthy();
  expect(incidentId).toEqual(expect.any(String));
});

it('RECOVERS when the retry button is pressed and the cause is gone', async
() => {
  const user = userEvent.setup();
  function Host(): React.ReactElement {
    const [broken, setBroken] = React.useState(true);
    return (
      <>
        <button type="button" onClick={() => setBroken(false)}>Fix
data</button>
        <ErrorBoundary tier="widget" fallback={s => <WidgetFallback {...s}
/>}>
          <Boom shouldThrow={broken} />
        </ErrorBoundary>
      </>
    );
  }

```

```

        </>
    );
}
render(<Host />);
expect(screen.getByRole('alert')).toBeInTheDocument();

await user.click(screen.getByRole('button', { name: 'Fix data' }));
await user.click(screen.getByRole('button', { name: /retry loading this
tile/i }));

expect(screen.getByText('Chart is fine')).toBeInTheDocument();
expect(screen.queryByRole('alert')).not.toBeInTheDocument();
});

it('resets automatically when a resetKey changes (navigation)', () => {
    const { rerender } = render(
        <ErrorBoundary tier="route" fallback={s => <WidgetFallback {...s} />}
resetKeys={['/a']}>
        <Boom shouldThrow />
    </ErrorBoundary>,
    );
    expect(screen.getByRole('alert')).toBeInTheDocument();

    rerender(
        <ErrorBoundary tier="route" fallback={s => <WidgetFallback {...s} />}
resetKeys={['/b']}>
        <Boom shouldThrow={false} />
    </ErrorBoundary>,
    );
    expect(screen.getByText('Chart is fine')).toBeInTheDocument();
});

it('does NOT catch errors thrown from an event handler', async () => {
    const user = userEvent.setup();
    const onError = jest.fn();
    render(
        <ErrorBoundary tier="widget" fallback={s => <WidgetFallback {...s} />}
onError={onError}>
        <button type="button" onClick={() => { try { throw new
Error('handler'); } catch { /* handled */ } }}>
            Click
        </button>
    </ErrorBoundary>,
    );
    await user.click(screen.getByRole('button', { name: 'Click' }));
    expect(onError).not.toHaveBeenCalled(); // documents the boundary's

```

```

limits
  });
});

// ===== 2. tenantSlice: normalization and selectors =====

const makeStore = () =>
  configureStore({ reducer: { tenant: tenantSlice.reducer } });

const user = (over: Partial<TenantUser> & { id: string; name: string }):
TenantUser => ({
  email: `${over.id}@acme.test`,
  roleIds: [],
  active: true,
  ...over,
});

const graph = {
  users: [
    user({ id: 'u-2', name: 'Bob', roleIds: ['r-admin'] }),
    user({ id: 'u-1', name: 'Alice', roleIds: ['r-admin', 'r-billing'] }),
    user({ id: 'u-3', name: 'Carol', active: false }),
  ],
  roles: [
    { id: 'r-admin', name: 'Admin', permissionIds: ['p-read', 'p-write'] },
    { id: 'r-billing', name: 'Billing', permissionIds: ['p-read', 'p-
invoice'] },
  ],
  permissions: [
    { id: 'p-read', key: 'tenant:read', description: 'Read' },
    { id: 'p-write', key: 'tenant:write', description: 'Write' },
    { id: 'p-invoice', key: 'billing:invoice', description: 'Invoice' },
  ],
};

describe('tenantSlice normalization', () => {
  it('stores each user exactly ONCE, keyed by id', () => {
    const store = makeStore();
    store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
    const state = store.getState();
    expect(usersSelectors.selectTotal(state)).toBe(3);
    expect(usersSelectors.selectById(state, 'u-1')?.name).toBe('Alice');
    // ids sorted by the adapter's comparer, not by payload order
    expect(usersSelectors.selectIds(state)).toEqual(['u-1', 'u-2', 'u-3']);
  });
});

```

```

it('updates one entity without touching the others (CRUD)', () => {
  const store = makeStore();
  store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
  const before = usersSelectors.selectById(store.getState(), 'u-2');

  store.dispatch(userUpdated({ id: 'u-1', changes: { name: 'Alice Cooper' } }));
  const state = store.getState();

  expect(usersSelectors.selectById(state, 'u-1').name).toBe('Alice Cooper');
  // Untouched entity keeps its identity: structural sharing works
  expect(usersSelectors.selectById(state, 'u-2')).toBe(before);
});

it('removes an entity from both ids and entities', () => {
  const store = makeStore();
  store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
  store.dispatch(userRemoved('u-3'));
  const state = store.getState();
  expect(usersSelectors.selectTotal(state)).toBe(2);
  expect(usersSelectors.selectById(state, 'u-3')).toBeUndefined();
  expect(usersSelectors.selectIds(state)).not.toContain('u-3');
});

it('does NOT mutate the previous state object', () => {
  const store = makeStore();
  store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
  const prev = store.getState().tenant;
  store.dispatch(userUpdated({ id: 'u-1', changes: { active: false } }));
  expect(store.getState().tenant).not.toBe(prev); // new reference
  expect(prev.users.entities['u-1'].active).toBe(true); // old snapshot intact
});

it('sets status and error across the thunk lifecycle', () => {
  const store = makeStore();
  store.dispatch(fetchTenantGraph.pending('req', 't-1'));
  expect(store.getState().tenant.status).toBe('loading');

  store.dispatch(fetchTenantGraph.rejected(null, 'req', 't-1', 'Boom'));
  expect(store.getState().tenant.status).toBe('failed');
  expect(store.getState().tenant.error).toBe('Boom');
});

```

```

describe('memoized selectors', () => {
  it('returns the SAME reference when inputs are unchanged', () => {
    const store = makeStore();
    store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
    const first = selectActiveUsers(store.getState());
    const second = selectActiveUsers(store.getState());
    expect(second).toBe(first); // identity preserved: no spurious re-
render
  });

  it('does not recompute when an UNRELATED slice value changes', () => {
    const store = makeStore();
    store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
    const before = selectActiveUsers(store.getState());
    selectActiveUsers.resetRecomputations();

    store.dispatch(fetchTenantGraph.pending('req2', 't-1')); // status only
    const after = selectActiveUsers(store.getState());

    expect(after).toBe(before);
    expect(selectActiveUsers.recomputations()).toBe(0);
  });

  it('DOES recompute when the users entity map changes', () => {
    const store = makeStore();
    store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
    selectActiveUsers(store.getState());
    selectActiveUsers.resetRecomputations();

    store.dispatch(userUpdated({ id: 'u-1', changes: { active: false } }));
    const after = selectActiveUsers(store.getState());

    expect(selectActiveUsers.recomputations()).toBe(1);
    expect(after).toHaveLength(1);
  });

  it('projects a primitive that never changes identity', () => {
    const store = makeStore();
    store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
    expect(selectActiveUserCount(store.getState())).toBe(2);
  });

  it('resolves effective permissions across roles, deduplicated', () => {
    const store = makeStore();
    store.dispatch(fetchTenantGraph.fulfilled(graph, 'req', 't-1'));
    const perms = selectEffectivePermissions(store.getState(), 'u-1');
  });
}

```

```

    expect(perms.map(p => p.key).sort()).toEqual([
      'billing:invoice',
      'tenant:read',
      'tenant:write',
    ]); // p-read appears in both roles, counted once
  });

  it('returns an empty list for an unknown user rather than throwing', () => {
    const store = makeStore();
    expect(selectEffectivePermissions(store.getState(),
    'nobody')).toEqual([]);
  });
});

// ===== 3. Axios interceptor: concurrent 401s =====

describe('apiClient 401 refresh queue', () => {
  let apiMock: MockAdapter;
  let authMock: MockAdapter;

  beforeEach(() => {
    __resetAuthState();
    tokenStore.setAccessToken('stale-token');
    apiMock = new MockAdapter(apiClient);
    authMock = new MockAdapter(authClient);
  });

  afterEach(() => {
    apiMock.restore();
    authMock.restore();
  });

  it('refreshes ONCE for 12 simultaneous 401s and re-fires every request',
  async () => {
    let refreshCalls = 0;
    authMock.onPost('/auth/refresh').reply(() => {
      refreshCalls += 1;
      return [200, { accessToken: 'fresh-token' }];
    });

    // 401 while the token is stale; 200 once the fresh token is presented
    apiMock.onGet(/\/widgets\/\d+\/).reply(config => {
      const auth = config.headers?.Authorization;
      return auth === 'Bearer fresh-token'
        ? [200, { ok: true }]
        : [401, { message: 'Token expired' }];
    });
  });
});

```

```

});

const responses = await Promise.all(
  Array.from({ length: 12 }, (_, i) => apiClient.get(`/widgets/${i}`)),
);

// THE assertion this whole design exists for
expect(refreshCalls).toBe(1);
expect(responses).toHaveLength(12);
responses.forEach(res => expect(res.status).toBe(200));
});

it('re-fires queued requests with the NEW token, not the stale one', async
() => {
  authMock.onPost('/auth/refresh').reply(200, { accessToken: 'fresh-token'
});
  const seen: string[] = [];
  apiMock.onGet('/a').reply(config => {
    const auth = String(config.headers?.Authorization);
    seen.push(auth);
    return auth === 'Bearer fresh-token' ? [200, {}] : [401, {}];
  });
  apiMock.onGet('/b').reply(config => {
    const auth = String(config.headers?.Authorization);
    seen.push(auth);
    return auth === 'Bearer fresh-token' ? [200, {}] : [401, {}];
  });

  await Promise.all([apiClient.get('/a'), apiClient.get('/b')]);
  expect(seen.filter(a => a === 'Bearer fresh-token')).toHaveLength(2);
});

it('does NOT loop when the refreshed token is still rejected', async () =>
{
  let attempts = 0;
  authMock.onPost('/auth/refresh').reply(200, { accessToken: 'still-bad'
});
  apiMock.onGet('/widgets/1').reply(() => {
    attempts += 1;
    return [401, {}];
  });

  await expect(apiClient.get('/widgets/1')).rejects.toBeDefined();
  expect(attempts).toBe(2); // original + exactly one retry, then stop
});

```



```

it('rejects every queued waiter when the refresh itself fails', async () => {
  authMock.onPost('/auth/refresh').reply(401, { message: 'Refresh expired' });
  apiMock.onGet('/widgets/\\d+/').reply(401, {});

  const results = await Promise.allSettled([
    apiClient.get('/widgets/1'),
    apiClient.get('/widgets/2'),
    apiClient.get('/widgets/3'),
  ]);
  // None left pending – the failure path flushes the queue
  expect(results.every(r => r.status === 'rejected')).toBe(true);
});

it('releases the lock so a LATER 401 can refresh again', async () => {
  let refreshCalls = 0;
  authMock.onPost('/auth/refresh').reply(() => {
    refreshCalls += 1;
    return [200, { accessToken: `token-${refreshCalls}` }];
  });
  apiMock.onGet('/widgets/1').replyOnce(401, {});
  apiMock.onGet('/widgets/1').replyOnce(200, {});
  await apiClient.get('/widgets/1');

  apiMock.onGet('/widgets/2').replyOnce(401, {});
  apiMock.onGet('/widgets/2').replyOnce(200, {});
  await apiClient.get('/widgets/2');

  expect(refreshCalls).toBe(2); // lock released after the first cycle
});

it('does not attempt a refresh for a 403', async () => {
  let refreshCalls = 0;
  authMock.onPost('/auth/refresh').reply(() => { refreshCalls += 1; return [200, {}]; });
  apiMock.onGet('/admin').reply(403, { message: 'Forbidden' });

  await expect(apiClient.get('/admin')).rejects.toMatchObject({ status: 403 });
  expect(refreshCalls).toBe(0);
});

it('transforms a network failure into a typed ApiError', async () => {
  apiMock.onGet('/widgets/1').networkError();
  await expect(apiClient.get('/widgets/1')).rejects.toMatchObject({ kind:

```

```

'network' });
});

it('transforms a timeout into a typed ApiError', async () => {
  apiMock.onGet('/slow').timeout();
  await expect(apiClient.get('/slow')).rejects.toMatchObject({ kind:
'timeout' });
});

it('aborts cleanly and reports cancellation, not failure', async () => {
  const controller = new AbortController();
  apiMock.onGet('/widgets/1').reply(() => new Promise(() => {}));
  const promise = apiClient.get('/widgets/1', { signal: controller.signal
});
  controller.abort();
  await expect(promise).rejects.toMatchObject({ kind: 'cancelled' });
});
});

```

The assertion that defines this chapter is `expect(refreshCalls).toBe(1)`. Twelve concurrent 401s, one refresh call, twelve successful retries. A boolean flag passes the "only one refresh" half and fails the "twelve successful retries" half — which is why the test asserts both, and why the Part 2 stampede reaches production in codebases that test only the first.

Part 5 — Active recall flashcards

Review on a 1/3/7/21-day schedule. **Tier A** covers state-scaling invariants and error boundaries and gates MOD-07. **Tier B** is RTK normalization and interceptors. **Tier C** is generators and transport architecture.

#	Tier	Prompt	Answer
1	A	Why does <code>React.memo</code> fail to stop a Context re-render?	Context does not propagate through props. When the provider's value identity changes, every consumer re-renders regardless of its props.
2	A	What is Context actually for?	Dependency injection of values that rarely change — theme, locale, tenant id, auth identity. It is not a state manager.

#	Tier	Prompt	Answer
3	A	How do you cut a Context blast radius?	Split providers by volatility and memoize each value. Fast-changing state should not live in a context at all.
4	A	What can an error boundary NOT catch?	Event handlers, async callbacks, <code>setTimeout</code> , SSR, and errors thrown by the boundary itself. Render-phase errors in children only.
5	A	Which lifecycle method gets the telemetry call, and why?	<code>componentDidCatch</code> . <code>getDerivedStateFromError</code> runs in the render phase and must stay pure, so logging there fires twice under StrictMode.
6	A	Why must boundaries exist before code splitting?	A failed dynamic import throws during render. Without a boundary, one stale <code>index.html</code> after a deploy white-screens the app.
7	A	Name the three boundary tiers and what each protects.	Root shell (the app survives), route layout (navigation survives, catches chunk failures), widget tile (the other 39 tiles survive).
8	A	What makes a reducer pure, and why does it matter?	No I/O, no randomness, no <code>Date.now()</code> , no mutation. Purity is what makes the action log replayable — the property Redux is bought for.
9	B	Where does the Immer mutation illusion hold?	Only inside <code>createSlice</code> and <code>createReducer</code> . In a thunk, a selector or a plain reducer you are touching real state.
10	B	What does <code>createEntityAdapter</code> give you, and what earlier lesson is it?	The <code>{ ids, entities }</code> normalized shape and CRUD reducers — the <code>reduce</code> -built lookup from MOD-01.8, formalized.
11	B	Why does <code>useSelector(s => s.x.ids.map(...))</code> re-render constantly?	It allocates a new array every call. <code>useSelector</code> compares by reference, so every action from any slice looks like a change.

#	Tier	Prompt	Answer
12	B	What is the middleware signature, and which earlier concept is it?	<code>store => next => action => result</code> — currying from MOD-02.2 applied to the dispatch pipeline. Forgetting <code>return next(action)</code> silently drops every action.
13	B	Why is a boolean <code>isRefreshing</code> flag not enough?	It prevents a second refresh call but gives the other requests nowhere to wait, so they fail with 401 while a valid token arrives. You need a shared promise plus a queue.
14	B	What does <code>config._retry</code> prevent?	The infinite 401 loop. A retried request that 401s again is terminal — log out rather than refresh forever.
15	B	Why must the refresh call use a separate Axios instance?	Otherwise a failing refresh passes through the 401 interceptor and triggers a refresh, recursively.
16	B	Why does the unlock belong in <code>finally</code> ?	An unexpected throw would otherwise leave <code>refreshPromise</code> non-null forever, so every future 401 queues against a settled promise and the tab hangs.
17	C	Does <code>fetch</code> reject on a 500?	No. Only network failures reject. Check <code>response.ok</code> or you will parse an HTML error page.
18	C	When has a team outgrown hand-rolled server state?	When they are writing cache keys, storing <code>lastFetched</code> in Redux, or implementing refetch-on-focus. Adopt RTK Query or TanStack Query.
19	C	What is <code>yield</code> two-way communication?	<code>yield</code> sends a value out; the argument to <code>.next(v)</code> becomes the result of that <code>yield</code> expression inside the generator.
20	C	How does <code>async/await</code> relate to generators?	It is generators plus a scheduler — a generator driven by a runner that calls <code>.next()</code> each time a promise settles.

#	Tier	Prompt	Answer
21	C	What does a saga yield, and why does that matter?	Plain effect <i>descriptions</i> the middleware interprets. That makes sagas testable without mocks and cancellable mid-flight, which <code>async / await</code> cannot be.
22	C	<code>takeEvery</code> or <code>takeLatest</code> for a subscription?	<code>takeLatest</code> — it cancels the previous run. <code>takeEvery</code> accumulates one live watcher per dispatch, which is the leak from Part 2.

Part 6 — Enterprise production readiness checklist

No PR touching state, boundaries or transport merges until every applicable line is verified. Reviewers cite the line in the review comment.

Context splitting and render boundaries

- ☐ Every provider value is wrapped in `useMemo` with a correct dependency list; no inline object literals as `value`
- ☐ Callbacks placed in a context value are wrapped in `useCallback` with functional updaters
- ☐ Contexts are split by volatility; fast-changing state (search text, hover, scroll) is not in a context
- ☐ State and dispatch are separate contexts where consumers only need one of them
- ☐ Context default value is `undefined` with a throwing access hook, not a plausible-looking default
- ☐ No server-fetched data stored in Context as an ad-hoc cache
- ☐ Provider placement is as low in the tree as the consumers allow
- ☐ Re-render impact measured in the Profiler after any provider change

Error boundary hierarchies and telemetry

- ☐ Boundaries exist at all three tiers: root shell, route layout, widget tile
- ☐ Every lazy-loaded route sits inside a boundary that handles chunk-load failure with a reload affordance
- ☐ `getDerivedStateFromError` is pure; all telemetry is in `componentDidCatch`

- ☐ Every boundary reports to telemetry with an incident id, component stack, tier and tenant context
- ☐ Every fallback offers a recovery path — `reset` , `navigate`, or `reload` — never a dead end
- ☐ `resetKeys` include route and tenant so navigation clears a stale error automatically
- ☐ Error triage distinguishes render crash, 401, 403, 404, 500 and network failure; no single generic message
- ☐ Event-handler and async errors are caught locally, since no boundary will see them
- ☐ A global `unhandledrejection` handler exists for what escapes everything else
- ☐ Boundary behaviour verified in a production build, not only in development

Redux state normalization and selector purity

- ☐ Reducers are pure: no `Date.now()` , no `Math.random()` , no I/O, no dispatch
- ☐ No mutation outside a `createSlice` or `createReducer` draft — thunks, selectors and plain reducers never touch state directly
- ☐ Relational data is normalized with `createEntityAdapter` ; no entity duplicated across slices
- ☐ `RootState` and `AppDispatch` are inferred from the store; typed `useAppSelector` and `useAppDispatch` used everywhere
- ☐ No selector allocates a new array or object inline in a component; projections go through `createSelector`
- ☐ `createSelector` input selectors return stable references
- ☐ Parameterized selectors take the parameter as an input selector rather than closing over it
- ☐ Thunks are typed with `createAsyncThunk<Returned, Arg, { state: RootState }>` ; no `as RootState` casts
- ☐ Thunks pass `signal` through so a cancelled dispatch aborts the request
- ☐ Only genuine shared client state is in the store; form drafts, hover and modal state stay local
- ☐ Action types read as `domain/eventInPastTense`
- ☐ Serializability and immutability middleware remain enabled in development; DevTools disabled in production

Network interceptor thread-safety and token hygiene

- ☐ A configured Axios **instance** per API surface; the bare `axios` export is never used
- ☐ The refresh endpoint uses a separate instance that bypasses the 401 interceptor

- ☐ The refresh lock is a single module-scoped promise, and concurrent 401s queue against it
- ☐ A concurrency test proves the refresh endpoint is called **exactly once** for N simultaneous 401s
- ☐ Every queued waiter is resolved or rejected on both the success and failure paths; none can stay pending
- ☐ `_retry` is set before re-firing; a second 401 on a retried request logs out rather than looping
- ☐ The lock is released in a `finally` block
- ☐ Access tokens are never persisted to `localStorage`; refresh tokens live in `HttpOnly`, `Secure`, `SameSite` cookies (MOD-02.9)
- ☐ Token store is cleared on logout, on refresh failure and on tenant switch
- ☐ Errors are transformed into a typed union at the interceptor; components never touch `error.response?.data`
- ☐ Correlation id attached to every request and surfaced in error fallbacks
- ☐ No `Authorization` header can reach a third-party domain
- ☐ Every request has a timeout; retries use exponential backoff and never retry 4xx

WCAG 2.1 AA error recovery and announcements

- ☐ Every error fallback carries `role="alert"` so it is announced on appearance (SC 4.1.3)
- ☐ Every retry affordance is a real `<button>`, keyboard-focusable and operable by Enter and Space (SC 2.1.1)
- ☐ Focus is moved to the fallback heading when a route-level boundary replaces page content (SC 2.4.3)
- ☐ Background refresh and refetch states announce through `role="status"` / `aria-live="polite"`, never assertively
- ☐ Refresh and retry announce once on settle, not per attempt
- ☐ Loading and error states are conveyed by text, not colour or a spinner alone (SC 1.4.1)
- ☐ Error messages name the problem and the action, and carry an incident id the user can quote (SC 3.3.1)
- ☐ A permission failure explains what is missing and who to contact, rather than only "Forbidden"
- ☐ Optimistic updates that roll back announce the rollback; the UI never silently reverts
- ☐ Fallback contrast meets 4.5:1; error states do not rely on red alone

Jest / RTL state and transport verification

- ☐ Slice reducers tested as pure functions: dispatch an action, assert the next state, assert the previous state is unchanged
- ☐ Entity adapter CRUD covered: add, update, remove, upsert, `setAll` replacement
- ☐ Selector memoization asserted with `recomputations()` — unrelated state changes must not recompute
- ☐ Selector reference stability asserted with `toBe`, not `toEqual`
- ☐ Thunk lifecycle tested across pending, fulfilled and rejected
- ☐ Error boundary tested for capture, accessible fallback, telemetry payload, retry recovery and `resetKeys` reset
- ☐ A test documents that boundaries do not catch event-handler errors
- ☐ Interceptor tested with **concurrent** 401s asserting one refresh call and N successful retries
- ☐ Interceptor failure path tested: every queued waiter rejects, nothing stays pending
- ☐ Retry-loop prevention tested: a persistently invalid token produces exactly one retry
- ☐ Error transformation tested for http, network, timeout and cancellation
- ☐ Module-scoped auth state reset between specs so lock state cannot leak
- ☐ Queries use `getByRole` / `getByLabelText`; no test IDs for user-facing elements
- ☐ Branch coverage at or above the project gate (90% for the data layer), enforced in CI

Exit criteria for MOD-06. LAB-06 passes the full suite, including the twelve-concurrent-401 test asserting exactly one refresh call, the selector `recomputations()` assertions, and the boundary recovery test; everything compiles under `strict` with zero `any`; all eight Tier A cards answer cold; and the learner can explain, without notes, why a boolean flag is not a refresh queue and why error boundaries must ship before code splitting. Only then does MOD-07 begin.